
Application Note : Development Manual for Telink BLE Mesh iOS SDK

AN-18051800-E1

Ver 1.0.1

2018/5/18

Brief:

This document is the guide for Telink BLE Mesh iOS SDK development.



TELINK SEMICONDUCTOR

**Published by
Telink Semiconductor**

**Bldg 3, 1500 Zuchongzhi Rd,
Zhangjiang Hi-Tech Park, Shanghai, China**

**© Telink Semiconductor
All Right Reserved**

Legal Disclaimer

Telink Semiconductor reserves the right to make changes without further notice to any products herein to improve reliability, function or design. Telink Semiconductor disclaims any and all liability for any errors, inaccuracies or incompleteness contained herein or in any other disclosure relating to any product.

Telink Semiconductor does not assume any liability arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others

The products shown herein are not designed for use in medical, life-saving, or life-sustaining applications. Customers using or selling Telink Semiconductor products not expressly indicated for use in such applications do so entirely at their own risk and agree to fully indemnify Telink Semiconductor for any damages arising or resulting from such use or sale.

Information:

For further information on the technology, product and business term, please contact Telink Semiconductor Company (www.telink-semi.com).

For sales or technical support, please send email to the address of:

telinknsales@telink-semi.com

telinknsupport@telink-semi.com

Revision History

Version	Major Changes	Date	Author
1.0.0	Initial release	2017/7	Shiqinglu, Cynthia
1.0.1		2018/5	Liangjiazhi

Table of contents

1	Overview.....	4
2	Telink Mesh Working Flow	5
3	SDK Development Guide	7
	1)Static method to generate Singleton manager:.....	7
	2)Bluetooth manager center initialization:.....	7
	3)Initiate scanning request:.....	8
	4)Connection.....	8
	5)Search for service characteristic list.....	9
	6>Login module:.....	9
	7)Data analysis:.....	9
	8)Other APIs:.....	10
	9)Command customization:.....	11
	10)Encryption and decryption:.....	12
	11)Detailed OTA process:.....	18
	12)Normal Add Device way: Normal Add.....	19
	13)New Add Device way: Mesh Add.....	19
	14)New Device OTA way: Mesh OTA.....	20
4	Appendix: Related document	21

Table of figures

Figure 1	Telink Mesh working flow	5
Figure 2	Packet sniff sample	12

1 Overview

Telink Mesh is wireless network with single BLE connection and multiple BLE (Bluetooth Low Energy) devices based on mesh communication protocol. To identify mesh network, each device uses network attribute, which contains mesh name, mesh password and ltk (long term key). Except that ltk is recommended to set as the default value, the other attribute parameters are modifiable.

The Mesh attribute strengthens the privacy of login. Each network corresponds to a login license: The factory default mesh name and password are “telink-mesh1” and “123”, respectively; while “ltk” is used for communication process. When the matched device is connected, it will request for login. The device won’t be controllable via command unless login succeeds.

The Mesh network with multi-hop relay supports wider communication range than Bluetooth. Device address, i.e. “u_DevAdress” obtained via “Online Status notify”, is used as unique identification of a device during communication process, and it’s also modifiable. To manage device, it’s recommended to modify device mesh information including mesh name and password, and set suitable “u_DevAdress” for each device.

2 Telink Mesh Working Flow

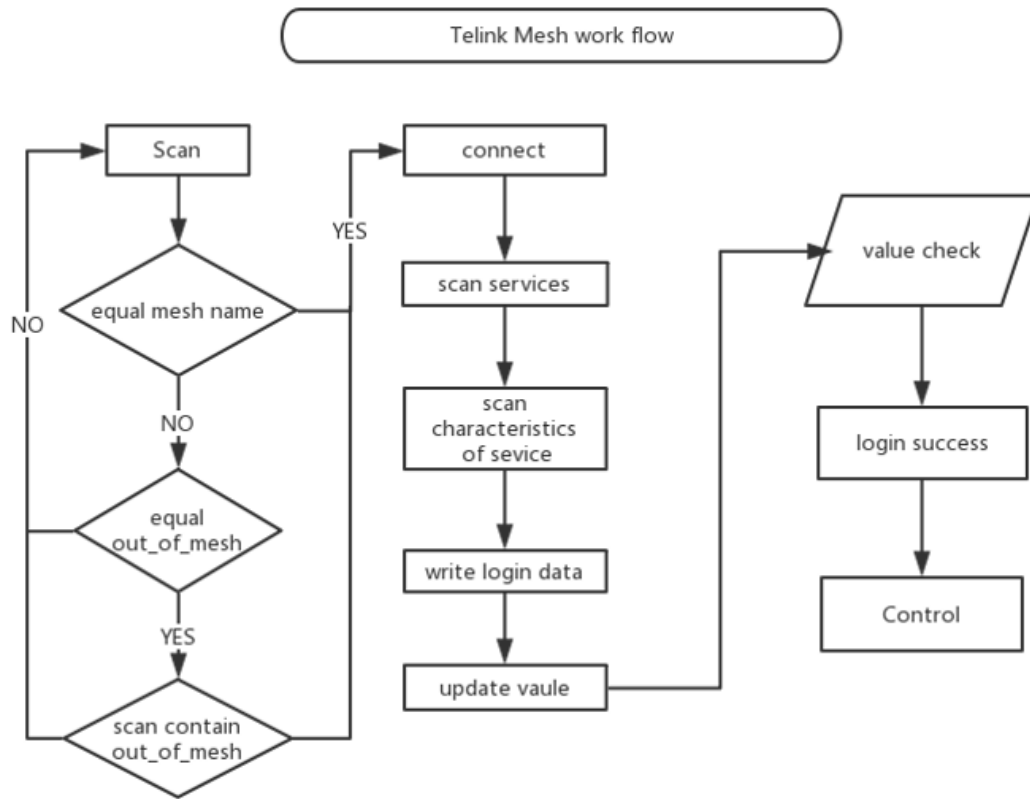


Figure 1 Telink Mesh working flow

*Notes:

- 1) During scanning, the delivered mesh name will be used for filtering. Since key is invisible in advertising packet, it cannot be checked. The matched device with consistent mesh name will be available after scanning.
- 2) If parameters match the license, the device will automatically be connected and then login the mesh network. After connection, it will automatically scan for Service ATT list and Characteristic list in Service. When the target characteristic is scanned, it will be written with login data and initiate login request by default. After successful login, it's able to control the device.
- 3) After successful login, the device will implement “-(void)setNotifyOpenPro”, so as to continuously request for “Online Status” for three times with 500ms interval.

“u_DevAdress”, “light_Brightness” and “light_Stata” of each device in Mesh network are all obtained by using this method; while “u_DevAdress” is used as unique identification to initiate control command.

3 SDK Development Guide

In SDK, a proprietary attribute “CBCentralManager” in a Singleton Class “BTCentralManager” is used as Bluetooth Manager Center, which constitutes Bluetooth Mesh along with peripherals. When this proprietary attribute is initialized, and this Singleton Class is set as manager center agent, Bluetooth “on”/“off” status will be checked. If it’s “on” status, a member variable parameter “isNeedScan” is supplied for SDK user to select whether scanning is needed. If this parameter is “YES”, signal scanning is started, and vice versa.

1) Static method to generate Singleton manager:

A Singleton will be generated to control callback of the whole agent.

```
+ (BTCentralManager*) shareBTCentralManager {
    static BTCentralManager *shareBTCentralManager = nil;
    static dispatch_once_t tempOnce=0;
    dispatch_once(&tempOnce, ^{
        shareBTCentralManager = [[BTCentralManager alloc] init];
        [shareBTCentralManager initData];
    });
    return shareBTCentralManager;
}
```

2) Bluetooth manager center initialization:

During initialization, Bluetooth status will be checked, and it will callback the agency method below to inform the SDK user of “centralManager” change.

```
- (void)centralManagerDidUpdateState:(CBCentralManager *)central {
    _centerState=central.state;
    //whether central's state is on
    if (central.state == CBCentralManagerStatePoweredOn) {
        if (isNeedScan)
            [self startScanWithName:self.userName Pwd:self.userPassword];
    } else if (central.state==CBCentralManagerStatePoweredOff){
        [self stopConnected];
        [self stopScan];
    }
}
```



```
//call back state of central
if (_delegate && [_delegate respondsToSelector:@selector(OnCenterStatusChange:)]) {
    [_delegate OnCenterStatusChange:self];
}
}
```

3) Initiate scanning request:

After Bluetooth manager center is established, Bluetooth stays in “on” status, and SDK user can use this method to scan for devices.

- (void)startScanWithName:(NSString *)nStr Pwd:(NSString *)pwd AutoLogin:(BOOL)autoLogin;
Scan callback, when device is discovered, callback the method below and corresponding parameters.

- (void)centralManager:(CBCentralManager *)central didDiscoverPeripheral:(CBPeripheral *)peripheral advertisementData: (NSDictionary<NSString *,id> *)advertisementData RSSI:(NSNumber *)RSSI

When advertising information of the matched device is obtained, the model “BTDevice” is used to store the received info in “_srcDevArrs”.

Data structure: <11021102 2211ffff 11022211 ffff0500 010f0000 01020304 05060708 090a0b0c 0d0e0f>

- (void)scanResult:(BTDevItem *)item;//agency method
//flag is DevChangeFlag_Add
- (void)OnDevChange:(id)sender Item:(BTDevItem *)item Flag:(DevChangeFlag)flag;

4) Connection

Once Bluetooth detects device, and initiates connection request, it may callback one of the following cases:

Connection success

- (void)centralManager:(CBCentralManager *)central didConnectPeripheral:(CBPeripheral *)peripheral

Connection termination

- (void)centralManager:(CBCentralManager *)central didFailToConnectPeripheral:(CBPeripheral *)peripheral error:(NSError *)error

Connection failure

- (void)centralManager:(CBCentralManager *)central didDisconnectPeripheral:(CBPeripheral *)peripheral error:(NSError *)error

And it will also callback the following agent:

-(void)OnDevChange:(id)sender Item:(BTDevItem *)item Flag:(DevChangeFlag)flag;

Related API:

-(void)connectWithItem:(BTDevItem *)citem

5) Search for service characteristic list

When device services are found, uuid is used to subscribe characteristics in services and store the target characteristics.

-(void)peripheral:(CBPeripheral *)peripheral didDiscoverServices:(NSError *)error
-(void)peripheral:(CBPeripheral *)peripheral didDiscoverCharacteristicsForService:(CBService *)service error:(NSError *)error

6) Login module:

When the characteristic of the target login operation is obtained, login operation can be implemented.

-(void)loginWithPwd:(NSString *)pStr;

//After characteristic is written with data, if there's corresponding response from device, it will callback the following API.

-(void)peripheral:(CBPeripheral *)peripheral didUpdateValueForCharacteristic:(CBCharacteristic *)characteristic error:(NSError *)error

7) Data analysis:

-(void)pasterData:(uint8_t *)buffer IsNotify:(BOOL)isNotify;

When "online status" is started, and there's corresponding notify, the agency method will be used to callback the obtained model.

-(void)notifyBackWithDevice:(DeviceModel *)model;

The method below applies to the case with feature "UpdateValue" returned:

-(void)OnDevNotify:(id)sender Byte:(uint8_t *)byte;// notifyFeature update

- (void)OnDevCommandReport:(id)sender Byte:(uint8_t *)byte;// [commandFeature update](#)

Please refer to 4 “Appendix: Related document” for analysis of the decrypted data.

8) Other APIs:

Set new network -> Set suitable mesh name and password, and set ltk as default value.

```
uint8_t tlkBuffer[20]=  
{0xc0,0xc1,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xd8,0xd9,0xda,0xdb,0xdc,0xdd,0xde,0xdf,0x0,0x0,0x0,  
0x0};
```

There are three similar methods, as shown below:

```
- (void)setNewNetworkName:(NSString *)nName Pwd:(NSString *)nPwd ltkBuffer:(uint8_t *)buffer;  
- (void)setNewNetworkName:(NSString *)nName Pwd:(NSString *)nPwd WithItem:(BTDevItem *)item ltkBuffer:(uint8_t *)buffer;  
- (void)setOut_Of_MeshWithName:(NSString *)addName PassWord:(NSString *)addPassWord  
NewNetWorkName:(NSString *)nName Pwd:(NSString *)nPwd ltkBuffer:(uint8_t *)buffer  
ForCertainItem:(BTDevItem *)item;
```

The configuration methods above contain connection login. Before connection login it will be marked as “configure network” first; after successful login, the following proprietary method will be used so as to implement actual configuration, and command will be sent to inform device to modify network.

```
- (void)setNewNetworkDataPro;// proprietary method
```

After successful configuration, it will callback success indication, and “pairFeature” will call “updatevalue back”.

```
- (void)sendPack:(NSData *)data; //send ota packet  
- (void)readFeatureOfselConnectedItem;//obtain attribute of directly connected light  
- (void)stopConnected;
```

This is a Proxy method,that call back when error in some situations.

```
- (void)exceptionReport:(int)stateCode errorCode:(int)errorCode deviceId:(int)deviceId;
```

This is a Proxy method, that call back when error in some situations.

- (void)loginTimeout:(TimeoutType)type;

9) Command customization:

The method below applies to all commands.

- (void)sendCommand:(uint8_t *)cmd Len:(int)len;

Please refer to 4 "Appendix: Related document".

Command samples:

- * turn on/off all peripherals in mesh
- * turn on/off single peripheral
- * turn off/on Group peripheral

- (void)turnOffAllLight;
- (void)turnOnAllLight;
- (void)turnOnCertainLightWithAddress:(uint32_t)u_DevAddress;
- (void)turnOffCertainLightWithAddress:(uint32_t)u_DevAddress;
- (void)turnOffCertainGroupWithAddress:(uint32_t)u_GroupAddress;
- (void)turnOnCertainGroupWithAddress:(uint32_t)u_GroupAddress;

- * add / delete to group

- (void)addDevice:(uint32_t)targetDeviceAddress
ToDestinateGroupAddress:(uint32_t)groupAddress;
- (void)deleteDevice:(uint32_t)deviceAddress ToDestinateGroupAddress:(uint32_t)groupAddress;

- * set luminance of peripheral in group or single

- (void)setLightOrGroupLumWithDestinateAddress:(uint32_t)destinateAddress
WithLum:(NSInteger)lum;

- * setting RGB of peripheral

- * @param destinateAddress address of single peripheral or group peripherals

- (void)setLightOrGroupRGBWithDestinateAddress:(uint32_t)destinateAddress
WithColorR:(float)R WithColorG:(float)G WithB:(float)B;

- * kick out peripheral (or peripherals, for group edit recommendation)

- * reset all parameters (like ltk/password/) of peripheral to the state of factory set
- * and mesh name is reset "out_of_mesh"

- (void)kickoutLightFromMeshWithDestinateAddress:(uint32_t)destinateAddress;

- * set CT(0~1) value of peripheral

- (void)setCTOfLightWithDestinationAddress:(uint32_t)destinationAddress AndCT:(float)CT;

10) Encryption and decryption:

Pnbr.	Time (us)	Channel	Access Address	Direction	ACK Status	Data Type	Data Header	L2CAP Length	ChanId	Opcode	AttHandle	AttValue	ATT_Write_Req
1189	+29769 =34188271	0x21	0xA6A65A66	M->S	OK	L2CAP-S	LLID NESH SN MD PDU-Length 2 1 1 0 24	0x0014	0x0004	0x12	0x0013	0C 1A0 A1 A2 A3 A4 A5 A6 A7 A8 02 CF 2D 20 30 E4 C9	
1190	+423 =34188694	0x21	0xA6A65A66	S->M	OK	Empty PDU	LLID NESH SN MD PDU-Length 1 0 1 0 0	0x8FE90F	-38	OK			randommm sk低8Byte
1191	+28578 =34218272	0x08	0xA6A65A66	M->S	OK	Empty PDU	LLID NESH SN MD PDU-Length 1 0 0 0 0	0x8FE4A9	-38	OK			Lower 8 bytes of sk
1192	+231 =34218503	0x08	0xA6A65A66	S->M	OK	L2CAP-S	LLID NESH SN MD PDU-Length 2 1 0 0 5	0x0001	0x0004	0x13	0x741036	0x741036	
1193	+29769 =34248272	0x14	0xA6A65A66	M->S	OK	L2CAP-S	LLID NESH SN MD PDU-Length 2 1 1 0 7	0x0003	0x0004	0x0A	0x0013	0xF3682A	
1194	+287 =34248559	0x14	0xA6A65A66	S->M	OK	Empty PDU	LLID NESH SN MD PDU-Length 1 0 1 0 0	0x8FE90F	-38	OK			Lower 8 bytes of sk
1195	+29713 =34278272	0x20	0xA6A65A66	M->S	OK	Empty PDU	LLID NESH SN MD PDU-Length 1 0 0 0 0	0x8FE4A9	-38	OK			randoms sk低8Byte
1196	+232 =34278504	0x20	0xA6A65A66	S->M	OK	L2CAP-S	LLID NESH SN MD PDU-Length 2 1 0 0 22	0x0012	0x0004	0x0B	0D BB E2 53 1F C7 D1 F3	0D BB E2 53 1F C7 D1 F3	

Figure 2 Packet sniff sample

During each login, it's needed to generate an 8-byte random number which is represented with "Byte[8]".

```
+ (BOOL)getRandPro:(uint8_t *)prand Len:(int)len {
    srand((int)time(0));
    memset(prand, 0, len);
    for (int i=0;i<len;i++)
        prand[i]=(uint8_t)random(255);
    return YES;
}
```

A "sk" is generated according to the three parameters including "meshName", "password" and "loginRand". The "loginRand" and lower 8 bytes of the generated "sk" (for check use) will be sent to module.

Please follow the method below to change "aes_att_er (meshName, password, loginRand, pcmd + 1)" as "oc" (Object_C) format:

```
+ (BOOL)encryptPair:(NSString *)uName Pas:(NSString *)uPas Prand:(uint8_t *)prand
PResult:(uint8_t *)result {
    uint8_t *tmpNetworkName = (uint8_t *)[uName
cStringUsingEncoding:NSUTF8StringEncoding];
    uint8_t *tmpPassword = (uint8_t *)[uPas cStringUsingEncoding:NSUTF8StringEncoding];
```

```

unsigned char      pNetworkName[16];

unsigned char      pPassword[16];

memset(pNetworkName, 0, 16);

memset(pPassword, 0, 16);

memcpy(pNetworkName, tmpNetworkName, strlen((char *)tmpNetworkName));

memcpy(pPassword, tmpPassword, strlen((char *)tmpPassword));

unsigned char sk[16], d[16], r[16];

for (int i=0; i<16; i++) {

    d[i] = pNetworkName[i] ^ pPassword[i];

}

memcpy (sk, prand, 8);

memset (sk + 8, 0, 8);

aes_att_encryption (sk, d, r);

memcpy (presult, prand, 8);

memcpy (presult+8, r, 8);

if (!(memcmp (prand, presult, 16))) return YES;

return NO;

}

```

When mobile phone receives response from module, “sectionKey” needed for encryption and decryption will be generated based on response data, “meshName”, “password” and “loginRand”.

Please follow the method below to change “aes_att_get_sk (meshName, password, loginRand, data, sectionKey)” as “oc” (Object_C) format:

```

+ (BOOL)getSectionKey:(NSString *)uName Pas:(NSString *)uPas Prandm:(uint8_t *)prandm
Prands:(uint8_t *)prands PResult:(uint8_t *)presult {

    uint8_t *tmpNetworkName = (uint8_t *)uName
cStringUsingEncoding:NSUTF8StringEncoding];

    uint8_t *tmpPassword = (uint8_t *)uPas cStringUsingEncoding:NSUTF8StringEncoding];

```

```
unsigned char    pNetworkName[16];

unsigned char    pPassword[16];

memset(pNetworkName, 0, 16);

memset(pPassword, 0, 16);

memcpy(pNetworkName, tmpNetworkName, strlen((char *)tmpNetworkName));

memcpy(pPassword, tmpPassword, strlen((char *)tmpPassword));

unsigned char sk[16], d[16], r[16];

for (int i=0; i<16; i++) {

    d[i] = pNetworkName[i] ^ pPassword[i];

}

memcpy (sk, prandm, 8);

memcpy (sk + 8, prands, 8);

aes_att_encryption (d, sk, r);

memcpy (presult, r, 16);

return YES;

}
```

a) Encryption:

sec_ivm[8] data structure is shown as below:

```
// phone: command

// sec_ivm [0] = slave_mac_address[0];

// sec_ivm [1] = slave_mac_address[1];

// sec_ivm[2] = slave_mac_address [2];

// sec_ivm[3] = slave_mac_address [3];

// sec_ivm[4] = 1

// sec_ivm[5] = sno[0];

// sec_ivm[6] = sno[1];

// sec_ivm[7] = sno[2];
```

When the fixed format above applies to command buffer, the method below is used to implement encryption.

```
aes_att_encryption_packet(pair_sk, sec_ivm, buff + 13, 2, buff + 15, n - 15)
```

Parameters:

- ✧ pair_sk: sk obtained after successful login.
- ✧ sec_ivm: please refer to sec_ivm[8] data structure.
- ✧ buff + 13: Address of the first byte in “src” filed of the command to be sent.
- ✧ 2: fixed as 2.
- ✧ buff + 15: Address of the first byte in “dst” filed of the command to be sent.
- ✧ n - 15: Length of data to be encrypted, i.e. the total length starting from “dst” filed to command end.

For APP development, APP can view data starting from “sno”, i.e.

```
CMD-Status_All: u8 cmd[] = { 11,11,51,00,00,ff,ff,da,11,02,10};
```

Therefore:

- ✧ “buff+13” should equal starting address of “00 00”, i.e. cmd + 3.
- ✧ “buff+15” should equal starting address of “ff ff”, i.e. cmd + 5.
- ✧ “n-15” should equal total data length – 5, i.e. sizeof(cmd) – 5.

Please follow the method below to change as oc (Object_C) format:

```
[CryptoAction encryptionPpacket:sectionKey Iv:sec_ivm Mic:buffer+3 MicLen:2 Ps:buffer+5
Len:15];
+ (BOOL)encryptionPpacket:(uint8_t *)key Iv:(uint8_t *)iv Mic:(uint8_t *)mic MicLen:(int)mic_len
Ps:(uint8_t *)ps Len:(int)len {
    uint8_t e[16], r[16], i;
    ////////// calculate mic //////////
    memset (r, 0, 16);
    memcpy (r, iv, 8);
    r[8] = len;
    aes_att_encryption (key, r, r);
    for (i=0; i<len; i++) {
        r[i & 15] ^= ps[i];
        if ((i&15) == 15 || i == len - 1) {
            aes_att_encryption (key, r, r);
        }
    }
}
```



```

    }
}
for (i=0; i<mic_len; i++) {
    mic[i] = r[i];
}
////////// calculate enc //////////
memset (r, 0, 16);
memcpy (r+1, iv, 8);
for (i=0; i<len; i++) {
    if ((i&15) == 0) {
        aes_att_encryption (key, r, e);
        r[0]++;
    }
    ps[i] ^= e[i & 15];
}
return YES;
}

```

b) Decryption:

sec_ivs[8] data structure is shown as below:

```

// sec_ivs[0] = slave_mac_address[0];
// sec_ivs[1] = slave_mac_address[1];
// sec_ivs[2] = slave_mac_address[2];
// sec_ivs[3] = sno[0];
// sec_ivs[4] = sno[1];
// sec_ivs[5] = sno[2];
// sec_ivs[6] = src[0]; //Obtain from transmitted command, it's the first byte
following "sno".

// sec_ivs[7] = src[1]; // Obtain from transmitted command, it's the second byte
following "sno".

```

The method below is used to implement decryption.

```

aes_att_decryption_packet(pair_sk, sec_ivs, p + 14, 2, p + 16, p[2] - 10);

```

Parameters:

- ✧ pair_sk: sk obtained after successful login.
- ✧ sec_ivs: please refer to sec_ivs[8] data structure.
- ✧ p + 14: Address of the first byte in “dst” field of the command to be decrypted.
- ✧ 2: fixed as 2.
- ✧ p + 16: Address of the first byte in “op code” field of the command to be decrypted.
- ✧ p[2] – 10: Length of data to be decrypted, i.e. the total length starting from “op code” field to command end.
- ✧ p[2]: L2cap_length of the received command.

For APP development, APP can view data starting from “sno”, i.e.

CMD-Status_All_Response:

```
u8 response[] = { 11,11,51,02,00, 02,00,db,11,02,ff,ff,ff,ff,ff,ff,00,00,04,01 }
```

Therefore:

- ✧ “p + 14” should equal the starting address of the second “02 00”, i.e. response+5.
- ✧ “p + 16” should equal the address of “db”, i.e. response+7.
- ✧ “p[2] - 10” should equal total length – 7, i.e. sizeof(response) – 7.

Please follow the method below to change as oc (Object_C) format:

```
[CryptoAction decryptionPpacket:sectionKey Iv:sec_ivm Mic:buffer+5 MicLen:2 Ps:buffer+7
```

```
Len:13];
```

```
+ (BOOL)decryptionPpacket:(uint8_t *)key Iv:(uint8_t *)iv Mic:(uint8_t *)mic MicLen:(int)mic_len
```

```
Ps:(uint8_t *)ps Len:(int)len {
```

```
    uint8_t e[16], r[16], i;
```

```
    ////////// calculate enc //////////
```

```
    memset (r, 0, 16);
```

```
    memcpy (r+1, iv, 8);
```

```
    for (i=0; i<len; i++) {
```

```
        if ((i&15) == 0) {
```

```
            aes_att_encryption (key, r, e);
```

```

        r[0]++;

    }

    ps[i] ^= e[i & 15];

}

////////// calculate mic //////////

memset (r, 0, 16);

memcpy (r, iv, 8);

r[8] = len;

aes_att_encryption (key, r, r);

for (i=0; i<len; i++) {

    r[i & 15] ^= ps[i];

    if ((i&15) == 15 || i == len - 1) {

        aes_att_encryption (key, r, r);

    }

}

for (i=0; i<mic_len; i++) {

    if (mic[i] != r[i]) {

        return NO;          //Failed

    }

}

return YES;

}

```

11) Detailed OTA process:

- ① You should call API “- (void)readFeatureOfselConnectedItem;” to get the version of device;SDK will back current version on Proxy method

“- (void)OnConnectionDevFirmWare:(NSData *)data;”

and

“- (void)OnConnectionDevFirmWare:(NSData *)data Item:(BTDevItem *)item;”.

- ② You should compare two version,and call API “- (void)sendPack:(NSData *)data;”

to send OTA data.

- ③ **Demo process:** send 16 packets OTA data, call API “-(void)readFeatureOfselConnectedItem;”, current version back, send next 16 packet OTA data. After all packet had sent, you should get current version and compare.

12) Normal Add Device way: Normal Add

a. You can constlt code file “AddDeviceViewController.m”.

b. Main Steps:

① scanDevice,API

```
“-(void)startScanWithName:(NSString *)nStr Pwd:(NSString *)pwd
AutoLogin:(BOOL)autoLogin;”
```

② changeAddress,API

```
“-(void)replaceDeviceAddress:(uint32_t)presentDevAddress
WithNewDevAddress:(NSUInteger)newDevAddress;”
```

③ changeMeshInfo,API

```
“-(void)setOut_Of_MeshWithName:(NSString *)addName PassWord:(NSString *)addPassWord
NewNetWorkName:(NSString *)nName Pwd:(NSString *)nPwd
ltkBuffer:(uint8_t *)buffer ForCertainItem:(BTDevItem *)item;”
```

④ (Optional) readFairwarm,API

```
“-(void)readFeatureOfselConnectedItem;”
```

c. **Attentions:**

- ① You should save addree at application, because addresses of mesh are unrepeatable.
- ② Address range:1-255

13) New Add Device way: Mesh Add

a. You can constlt code file “MeshAddVC.m”.

b. Main Steps:

① Enter default mesh,if application had not in mesh,use API

```

-(void)startScanWithName:(NSString *)nStr Pwd:(NSString *)pwd
AutoLogin:(BOOL)autoLogin;”

```

If application had in mesh,use API

```

- (void)allDefault;”

```

② scan device,API

```

- (void)getAddressMac”

```

③ change address,API

```

- (void)changeDeviceAddress:(uint8_t)address new:(uint8_t)newAddress
mac:(uint32_t)mac;”

```

④ change mesh info,API

```

- (void)updateMeshInfo:(NSString *)nname password:(NSString *)npasword;”

```

⑤ (Optional) recovery mesh,if you had used “allDefault” at step1,you need recovery mesh.API

```

- (void)allReset;”

```

c. Attentions:

- ① You should save addree at application,because addresses of mesh are unrepeatable.
- ② Address range:1-255
- ③ SDK will add a part of devices which had changed address to mesh.
- ④ if you had used “allDefault” at step1,do not forget recovery mesh at sept5.

14) New Device OTA way: Mesh OTA

a. You can constlt code file “MeshOTAVC.m”.

b. Main API:

① Set DataSource,API

```

- (void)setCurrentDevices:(NSArray <DeviceModel *>*)devices;”

```

② Start meshOTA when that is not in meshOTAing,API

```
“- (void)startMeshOTAWithDeviceType:(NSInteger)deviceType otaData:(NSData*)otaData  
progressHandle:(ProgressBlock)progressBlock  
finishHandle:(FinishBlock)finishBlock errorHandler:(ErrorBlock)errorBlock;”
```

③ **Continue meshOTA when that is in meshOTAing,API**

```
“- (void)continueMeshOTAWithDeviceType:(NSInteger)deviceType  
progressHandle:(ProgressBlock)progressBlock  
finishHandle:(FinishBlock)finishBlock errorHandler:(ErrorBlock)errorBlock”
```

c. Attentions:

- ① **You have to set up the data source.**
- ② **You don't need to use API “stopMeshOTA” only when user stop meshOTA.**
- ③ **You can't use API “startMeshOTA” or “continueMeshOTA” repeating only when meshOTA is complete.**

4 Appendix: Related document

- 1) AN_BLE-15120202_Communication Protocol for Telink BLE Mesh Light APP